

EXPERIMENT NO 9

EPSILON CLOSURE

NAME: K P ASHIL

CLASS: S 7 CSE

ROLL NO: 29

DATE: 09/10/2024

CODE:

```
#include<stdio.h>
#define MAX_STATES 100
#define MAX_SYMBOLS 100
typedef struct {

int transition[MAX_STATES][MAX_SYMBOLS][MAX_STATES];

int epsilon[MAX_STATES][MAX_STATES];

int numStates;

int numSymbols;

} NFA;

void addTransition(NFA *nfa, int from, int to, int symbol) {

nfa->transition[from][symbol][to] = 1;

}

void addEpsilonTransition(NFA *nfa, int from, int to) {

nfa->epsilon[from][to] = 1;

}

void epsilonClosure(NFA *nfa, int state, int *closure, int *closureSize) {

closure[*closureSize] = state;

(*closureSize)++;

// Check for epsilon transitions
```

```

for (int i = 0; i < nfa->numStates; i++) {
    if (nfa->epsilon[state][i]) {
        // Check if state is already in closure
        int alreadyInClosure = 0;
        for (int j = 0; j < *closureSize; j++) {
            if (closure[j] == i) {
                alreadyInClosure = 1;
                break;
            }
        }
        if (!alreadyInClosure) {
            epsilonClosure(nfa, i, closure, closureSize);
        }
    }
}

int main() {
    NFA nfa;

    int epsilonClosureResult[MAX_STATES];

    printf("Enter the number of states: ");
    scanf("%d", &nfa.numStates);

    printf("Enter the number of input symbols: ");
    scanf("%d", &nfa.numSymbols);

    for (int i = 0; i < nfa.numStates; i++) {
        for (int j = 0; j < nfa.numSymbols; j++) {

```

```

for (int k = 0; k < nfa.numStates; k++) {
    nfa.transition[i][j][k] = 0; // No transitions
}

}

for (int k = 0; k < nfa.numStates; k++) {
    nfa.epsilon[i][k] = 0; // No epsilon transitions
}

}

for (int i = 0; i < nfa.numStates; i++) {
    printf("\nEnter the transitions for state %d\n",i);
    for (int j = 0; j < nfa.numSymbols; j++) {
        printf("\t for symbol %c: ", j==nfa.numSymbols-1? 35 : j+97);
        int toState;
        while (1) {
            scanf("%d", &toState);
            if(toState == -1) break;
            if(j==nfa.numSymbols-1) //checking whether epsilon transition or normal input transition
                addEpsilonTransition(&nfa, i, toState);
            else
                addTransition(&nfa, i, toState, j);
        }
    }
}

for (int i = 0; i < nfa.numStates; i++) {

```

```

int closure[MAX_STATES];

int closureSize = 0;

epsilonClosure(&nfa, i, closure, &closureSize);

printf("Epsilon closure of state %d: { ", i);

for (int j = 0; j < closureSize; j++) {

printf("%d ", closure[j]);

}

printf("}\n");

}

return 0;

}

```

OUTPUT

Enter the number of states: 8
Enter the number of input symbols: 3

Enter the transitions for state 0
for symbol a: -1
for symbol b: -1
for symbol #: 1 7 -1

Enter the transitions for state 1
for symbol a: -1
for symbol b: -1
for symbol #: 2 4 -1

Enter the transitions for state 2
for symbol a: 3 -1
for symbol b: -1
for symbol #: -1

Enter the transitions for state 3
for symbol a: -1
for symbol b: -1
for symbol #: 6 -1

Enter the transitions for state 4
for symbol a: -1
for symbol b: 5 -1
for symbol #: -1

Enter the transitions for state 5
for symbol a: -1
for symbol b: -1
for symbol #: 6 -1

Enter the transitions for state 6
for symbol a: -1
for symbol b: -1
for symbol #: 1 7 -1

Enter the transitions for state 7
for symbol a: -1
for symbol b: -1
for symbol #: -1

Epsilon closure of state 0: { 0 1 2 4 7 }

Epsilon closure of state 1: { 1 2 4 }

Epsilon closure of state 2: { 2 }

Epsilon closure of state 3: { 3 6 1 2 4 7 }

Epsilon closure of state 4: { 4 }

Epsilon closure of state 5: { 5 6 1 2 4 7 }

Epsilon closure of state 6: { 6 1 2 4 7 }

Epsilon closure of state 7: { 7 }

EXPERIMENT NO 10

E-NFA TO DFA

NAME: K P ASHIL

CLASS: S 7 CSE

ROLL NO: 29

DATE: 16/10/2024

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<ctype.h>
#include<stdbool.h>
```

```
char states[100],terms[100],stack[50];
int isfinal[100],resfinal[100];
int ind=0,stacklen=0;
char grams[20];
char rhs[20][50];
int count=0,rescount=0,resind=0;
int sn=0,tn=0;
```

```
char resStates[100][20],restable[100][20][10];
```

```
char table[50][50][10];
char hasEps[100];
int eind=0;
```

```
char eclsr[50];
int clsind=0;
```

```
char sstr[30];
int sortString()
{
    int l=strlen(sstr);
    for(int i=0;i<l;i++)
    {
        for(int j=0;j<l-1-i;j++)
        {
            if(sstr[j]>sstr[j+1])
            {
                char tem=sstr[j];
                sstr[j]=sstr[j+1];
                sstr[j+1]=tem;
            }
        }
    }
}
```

```

int eclfnl=0;

int eClosure(char c)
{
    for(int i=0;i<sn;i++)
    {
        if(states[i]==c)
        {
            eclfnl=eclfnl|isfinal[i];
            if(strcmp(table[i][tn],"-")!=0)
            {
                strcat(eclsr,table[i][tn]);
                clsind+=strlen(table[i][tn]);
                for(int j=0;j<strlen(table[i][tn]);j++)
                {
                    eClosure(table[i][tn][j]);
                }
            }
            break;
        }
    }
    return 0;
}

```

```

int main()
{
    printf("Number of states\t");
    scanf("%d",&sn);
    printf("Number of terminals\t");
    scanf("%d",&tn);
    getchar();
    printf("Enter States\t");
    for(int i=0;i<sn;i++)
    {
        states[i]=getchar();
        getchar();
    }
    printf("Enter terminals\t");
    for(int i=0;i<tn;i++)
    {
        terms[i]=getchar();
        getchar();
    }
    printf("Transition table\n");
    for(int i=0;i<tn;i++)
    {
        printf("\t%c",terms[i]);
    }
    printf("\tEPSL\tFinal?\n");
    for(int i=0;i<sn;i++)
    {

```

```

printf("%c\t",states[i]);
for(int j=0;j<tn+2;j++)
{
    scanf("%s",table[i][j]);
    if(j==tn&&strcmp(table[i][j],"-")!=0)
    {
        hasEps[eind]=states[i];
        eind++;
    }
    if(j==tn+1)
    {
        if(strcmp(table[i][j],"y")==0)
        {
            isfinal[i]=1;
        }
        else
            isfinal[i]=0;
    }
}
}

for(int i=0;i<eind;i++)
{
    eclsr[0]='\0';
    clsind=0;
    eClosure(hasEps[i]);
    for(int j=0;j<sn;j++)
    {
        for(int k=0;k<tn;k++)
        {
            for(int l=0;l<strlen(table[j][k]);l++)
            {
                if(table[j][k][l]==hasEps[i])
                {
                    for(int p=0;p<clsind;p++)
                    {
                        int flag=1;
                        for(int o=0;o<strlen(table[j][k]);o++)
                        {
                            if(table[j][k][o]==eclsr[p])
                            {
                                flag=0;
                                break;
                            }
                        }
                        if(flag)
                        {
                            table[j][k][strlen(table[j][k])]=eclsr[p];
                        }
                    }
                    break;
                }
            }
        }
    }
}

```

```

        }
    }
}
int thisind=0,fin=0;
for(int k=0;k<sn;k++)
{
    for(int l=0;l<clsind;l++)
    {
        for(int p=0;p<sn;p++)
        {
            if(states[p]==eclsr[l]&&isfinal[p]==1)
            {
                fin=1;
                break;
            }
        }
        if(fin==1)
            break;
    }
    if(states[k]==hasEps[i])
        thisind=k;
}
for(int j=0;j<clsind;j++)
{
    for(int k=0;k<sn;k++)
    {
        if(eclsr[j]==states[k])
        {
            for(int l=0;l<tn;l++)
            {
                for(int o=0;o<strlen(table[k][l]);o++)
                {
                    int ffl=0;
                    for(int p=0;p<strlen(table[thisind][l]);p++)
                    {
                        if(table[thisind][l][o]==table[k][l][p])
                        {
                            ffl=1;
                            break;
                        }
                    }
                    if(ffl==0&&table[k][l][o]!='-')
                    {
                        if(table[thisind][l][0]!='-')
                            table[thisind][l][0]=table[k][l][o];
                        else
                            table[thisind][l][strlen(table[thisind][l])]=table[k][l][o];
                    }
                }
            }
        }
    }
}

```

```

        }
    }
    isfinal[thisind]=fin|isfinal[thisind];
}
printf("\nThe equivalent DFA\n");
eclsr[0]='\0';
clsind=0;
eclfnl=0;
if(strcmp(table[0][tn],"-")==0)
{
    eclsr[0]=states[0];
    eclsr[1]='\0';
    resfinal[0]=isfinal[0];
}
else
{
    eClosure(states[0]);
    resfinal[0]=eclfnl;
}
strcpy(resStates[0],eclsr);
for(int i=0;i<tn;i++)
{
    strcpy(restable[0][i],table[0][i]);
}
rescount=1;
resind=0;
char cures[50];
int curlen=0;
while(rescount>resind)
{
    for(int j=0;j<tn;j++)
    {
        int flgg=0;
        int ri=0;
        flgg=-1;
        curlen=0;
        for(int k=0;k<strlen(resStates[resind]);k++)
        {
            for(int l=0;l<sn;l++)
            {
                if(states[l]==resStates[resind][k])
                {
                    resfinal[resind]=resfinal[resind]|isfinal[l];
                    flgg=l;
                    break;
                }
            }
        }
        if(flgg>=0)
        {
            for(int p=0;p<strlen(table[flgg][j]);p++)
            {
                int fff=1;
                for(int l=0;l<curlen;l++)

```

```

        {

            if(curres[l]==table[flgg][j][p])
            {
                fff=0;
                break;
            }
        }
        if(fff==1&&table[flgg][j][p]!='-')
        {
            curres[curlen]=table[flgg][j][p];
            curlen++;
            resfinal[resind]=isfinal[flgg];
        }
    }
}
if(curlen==0)
continue;
flgg=1;
strcpy(sstr,curres);
sortString();
strcpy(curres,sstr);
for(int k=0;k<rescount;k++)
{
    curres[curlen]='\0';
    strcpy(sstr,resStates[k]);
    sortString();
    strcpy(resStates[k],sstr);
    if(strcmp(resStates[k],curres)==0)
    {
        flgg=0;
    }
}
if(flgg==1)
{
    strcpy(resStates[rescount],curres);
    rescount++;
}
strcpy(restable[resind][j],curres);
}
resind++;
}
for(int i=0;i<tn;i++)
{
    printf("\t%c",terms[i]);
}
printf("\tFinal?\n");
for(int i=0;i<rescount;i++)
{
    if(i==0)

```

```

        printf("Start state ");
    else if(i+1==rescount)
        printf("Final State ");
    else
        printf(" ");
        printf("%c\t",('A'+i));
        for(int j=0;j<tn+1;j++)
        {
            if(j==tn)
            {
                if(resfinal[i])
                {
                    printf("YES\n");
                }
                else
                    printf("NO\n");
            }
            else if(strcmp(restable[i][j],"")==0||strcmp(restable[i][j],"-")==0)
                printf("NULL\t");
            else
            {
                for(int k=0;k<rescount;k++)
                {
                    if(strcmp(resStates[k],restable[i][j])==0)
                    {
                        printf("%c\t",('A'+k));
                        break;
                    }
                }
            }
        }
    }
    printf("\n");
}

```

OUTPUT

Number of states 11

Number of terminals 2

Enter States 0

1

2

3

4

5

6

7

8

9

10

Enter terminals a

b

Transition table

	epsln	Final?
0	- - 17	n
1	- - 24	n
2	3 - -	n
3	- - 6	n
4	- 5 -	n
5	- - 6	n
6	- - 17	n
7	8 - -	n
8	- 9 -	n
9	- 10 -	n
1	- - -	y

The equivalent DFA

	Final?
Start state A	B C NO
	B B D NO
	C B C YES
	D B E NO
Final State E	B C YES

EXPERIMENT NO 11

LEXICAL ANALYZER USING LEX TOOL

NAME: K P ASHIL

CLASS: S 7 CSE

ROLL NO: 29

DATE: 23/10/2024

lex.l

```
%{
#include <stdio.h>
    int no=1;
}%
lite [0-9][0-9]*
inv [0-9][a-zA-Z0-9]*
ident [a-zA-Z][a-zA-Z0-9]*
lite2 \"[a-zA-Z][a-zA-Z0-9]*\"
rel <=|<|>|=|>
key int|void|char|if
%%
{lite} {printf("%s\t%i\tLiteral\n",yytext,no);}
{lite2} { char aa[10],bb[10];
        strcpy(aa,yytext);
        for(int i=1;i<strlen(aa)-1;i++){
            bb[i-1]=aa[i];
        }
        printf("%s\t%i\tLiteral\n",bb,no);
    }
{inv} {printf("Invalid:\t%s\n",yytext);}
{rel} {
    printf("%s\t%i\tRelational Operator, \t",yytext,no);
    if(strcmp(yytext,"<")==0){printf("LE");}
    if(strcmp(yytext,"<")==0){printf("LT");}
    if(strcmp(yytext,">")==0){printf("LE");}
    if(strcmp(yytext,">")==0){printf("LT");}
    printf("\n");
}
"+"|"-"|"*"|"/" {printf("%s\t%i\tArithmetic opeator, ",yytext,no);
    if(strcmp(yytext,"+")==0){printf("ADD");}
    if(strcmp(yytext,"-")==0){printf("SUB");}
    if(strcmp(yytext,"/")==0){printf("DIV");}
    if(strcmp(yytext,"*")==0){printf("MUL");}
    printf("\n");
}
"=" {printf("%s\t%i\tAssignment operator, EQ\n",yytext,no);}
```

```

{key} {printf("%s\t%i\tkeyword\n",yytext,no);}
{ident} { printf("%s\t%i\tIdentifier\n",yytext,no);}
"\n" {no++;}
. {}
%%
int main()
{
    yyin=fopen("input.txt","r");
    printf("Lexeme\tLine\tToken\n");
    yylex();
}

```

input.txt

```

void main(){

    int a,b,c=0;
    char ch;
    a=b+c;
    if(a<=b)
        a=b;
    printf("Hello World");
}

```

OUTPUT

```

sudo apt update
sudo apt install flex
lex pgm.l
gcc lex.yy.c -ll
./a.out<input.txt

```

Lexeme	Line	Token
void	1	keyword
main	1	Identifier
int	3	keyword
a	3	Identifier
b	3	Identifier
c	3	Identifier
=	3	Assignment operator, EQ
0	3	Literal
char	4	keyword
ch	4	Identifier
a	5	Identifier
=	5	Assignment operator, EQ
b	5	Identifier
+	5	Arithmetic opeator, ADD
c	5	Identifier
if	6	keyword

a	6	Identifier
<=	6	Relational Operator, LE
b	6	Identifier
a	7	Identifier
=	7	Assignment operator, EQ
b	7	Identifier
printf	8	Identifier
Hello	8	Identifier
World	8	Identifier

EXPERIMENT NO 12

SIMPLE CALCULATOR USING YACC

NAME: K P ASHIL

CLASS: S 7 CSE

ROLL NO: 29

DATE: 23/10/2024

pgm.l

```
%{
#include<stdio.h>
#include "y.tab.h"
extern int yylval;
}%
%%
[0-9]+ { yylval=atoi(yytext); return NUMBER;}
[\t] ;
[\n] return 0;
. return yytext[0];
%%
int yywrap()
{
return 1;
}
```

pgm.y

```
%{
#include<stdio.h>
int flag=0;
}%
%token NUMBER
%left '+' '-'
%left '*' '/' '%'
%left '(' ')'
%%
ArithmeticExpression: E {printf("output:%d\n",$$);return 0;};
E:E+'E' {$$=$1+$3;}
|E-'E' {$$=$1-$3;}
|E'*E' {$$=$1*$3;}
|E'/E' {$$=$1/$3;}
|E'%E' {$$=$1%$3;}
|'('E')' {$$=$2;}
|NUMBER {$$=$1;}
;
```

```
%%  
void main()  
{  
printf("\nEnter the input:\n");  
yyparse();  
if(flag==0)  
printf("Expression is Valid\n");  
}  
void yyerror()  
{  
printf("Expression is invalid\n");  
flag=1;  
}
```

OUTPUT

```
sudo apt update  
sudo apt install flex bison  
yacc -d pgm.y  
lex pgm.l  
gcc lex.yy.c y.tab.c -w  
./a.out
```

```
Enter the input:  
5*2  
output:10  
Expression is Valid
```